

Verification loop — measurement report

Everything in this document was paid for. Twenty-eight provider calls across nine campaigns, 2026-07-14 to 2026-07-16, on gpt-5.6-sol at medium effort. It records what we asked, what we got, and what we threw away.

The question

Does this environment make a coding agent better, or does it only make it more expensive?

The environment existed for months on the assumption that the answer was obvious. It was not.

Phase 1 — the null (six fixtures, 17 calls)

We built a five-level mode ladder: vanilla → mode-1-lean → mode-2-routed → mode-3-assured → full, each level adding scaffolding — routed knowledge modules, an objective ledger, an impact map, an adversarial threat model, an independent verifier. Then we measured it.

Fixture	Best score	Who achieved it
adaptive-contract-evolution-v2	100	every arm, including bare vanilla
adaptive-contract-evolution-v3	100	every arm
adaptive-contract-evolution-v4 (multi-hop propagation)	100	every arm; full cost 1.63M vs vanilla's 212k
implicit-conventions-v1 (underspecified)	100	every arm
implicit-conventions-scale-v1 (57 modules, 33k tokens)	100	every arm
medi-ny-parser-rework-v1 (real shipped fix)	89	bare vanilla, at 350k tokens

full scored 89 on the real task too — for 3.75M tokens. **10.7× the cost for an identical result.**

Two independent fixture designers, deliberately targeting the model's demonstrated weaknesses, could not build a fair task where scaffolding won. The conclusion was unavoidable and unwelcome: **prompt scaffolding is overhead**. Our own research corpus had predicted it, in writing, three days earlier — "strip scaffolding as models improve; each harness component encodes an expiring assumption."

We also learned the score itself lies at n=1: the same bare arm scored **73 on one run and 89 on the next**, failing different dimensions each time. Every verdict since then requires three trials per arm.

Phase 2 — the diagnosis

If scaffolding does not help, what would? The audit compared every research finding against its implementation and found the same defect five times: **findings implemented as forms instead of mechanisms**.

- "Run deterministic repo-wide symbol verification" became an impact-map *form the agent filled in*.
- "Verify with a separate evaluator" became a verifier whose verdict was *wired to nothing* — on the real task it reported *complete-quality-failure* and the run scored 89 anyway.
- "Engineer the loop, not the prompt" became *prompts*.

The proof was a defect called *paren-wrap*: a drug name split across two PDF lines. Every arm missed it — including the ones with the "completeness ledger", because the ledger compiles requirements from the *task text*, and this requirement existed only in the *data*. A control that only inspects the agent's own claims cannot catch what the agent never enumerated.

Phase 3 — the loop (11 calls, self-invalidated)

We rebuilt around forcing functions: a harness-authored check suite (acceptance, behavioural differential against a pre-change snapshot, deterministic symbol sweep, property checks over real data), executed by the harness, failures handed back, iterate to green under hard budgets.

The first campaign was **contaminated by our own mistake** — the fixture's checks were placed where every arm could read them, including the bare control. The result was a perfect flatline: nine runs, all scoring 88, all failing the same dimension in the same way. We were measuring the checks as *context*, not measuring the loop. Eleven calls, discarded.

It was not wasted. It taught three things:

1. **Check content steers hard.** Every arm suddenly fixed `paren-wrap` — the defect no arm had ever fixed. What you name, the model repairs.
2. **The protected-files tripwire works on a live agent.** One arm added tests to a file it was told not to touch; the run was invalidated automatically.
3. **A subtly mis-scoped check homogenises failures.** Our property rule had a blind spot (a name reduced to the token `mg` passed every regex) and every arm fell into the same trap. Checks need the same admission discipline as fixtures: the known-bad must fail, the reference must pass.

Phase 4 — the loop measured cleanly (11 calls)

Checks moved harness-side. Bare arms genuinely bare.

Arm	Scores	Tokens	What happened
vanilla	77, 62	~300k	Wild variance. Over-strips a drug name to 'mg' with no checks in sight — the defect is the model's own tendency, not something the checks induced.
vanilla-loop	77, 89 , 89	265–627k	Dose-response: converged → 89, cut off by the iteration cap → 77. The only run that recovered Cymalta did it immediately after feedback named the residue.
standard (heavyweight)	88, 88, 88	1.1–1.6M	Zero variance — and its gate refused to certify all three . The agents ran out of turns mid-fix: the budget went to evidence paperwork.

The loop lifted the bare agent ~12–15 points at half the scaffold's cost. Nobody reached 100. The heavyweight scaffold knew things the loop didn't (it read the conventions and fixed `paren-wrap`) but had no cheap way to iterate; the loop iterated cheaply but didn't know what the scaffold knew.

Phase 5 — the lean loop (14 calls, three trials per arm)

Merge the winners, cut the loser:

- **Harness-generated evidence** — the harness re-executes every check, so the harness writes the evidence. Agent paperwork: deleted.
- **Guided feedback** — a *failing* check carries the relevant convention excerpt and fix direction; a passing check carries nothing. Just-in-time retrieval, applied to verification.
- **Iteration ceiling 3** — the cap of 2 had cut a run off before it converged.

Arm	Scores	Total tokens per trial	Iterations
vanilla	77, 77, 77	272–316k	—
vanilla-loop (guided)	89 ¹ , 100 , 100	647k–1.21M	2, 2, 2
standard-loop (lean scaffold + guided loop)	100 , 100 , 100	1.54–2.68M	2, 2, 1

¹ Invalidated by the protected-files tripwire.

100 means the agent matched the fix a human engineer actually shipped, on their own proprietary code, from the same starting point and the same brief. Three trials, three times, zero variance, zero violations.

Guidance attached to failures fixed both remaining defects at once — the Cymbalta bleed row and the fentanyl patch preservation — the two that Phase 4's unguided loop and heavyweight scaffold had each missed separately.

What is true now

1. **Prompt scaffolding does not buy correctness.** Measured across six fixtures. Dead.
2. **An independently executed loop does.** 77 → 100, replicated three times on real work.
3. **Guidance belongs on failures, not in prompts.** It is the highest-leverage single component measured, and it costs nothing when the work is already green.
4. **Cost is claim-based.** Iterations happen only when a check is red — exactly when the bare agent would have shipped the defect. Green-first-iteration work costs an unscaffolded run plus seconds of CPU.
5. **A single trial measures nothing.** 16-point same-arm swings. Three trials minimum, always.

What is not true, and what we do not know

- **"≤2× the cost" was our hypothesis and it failed.** The loop-only path reached the ceiling at 2–4×; the lean scaffold path at 5–9×.
- **One task family.** The loop is measured on `medi-ny`. Replication elsewhere is the honest next step.
- **The scaffold's marginal contribution over the loop is unresolved.** Both reach 100; three trials cannot separate reliability differences that small.
- **Two research findings remain unmechanised:** end-to-end verification through real browser automation, and genuine adversarial multi-agent debate.

Incidents worth remembering

- **Stale bytecode nearly falsified everything.** A source file rewritten within the same second at the same size passes CPython's `(mtime, size).pyc` validation. A red check ran green on a stale compile. Found by a probe, not by a human reading code. The harness now purges bytecode before every run.
- **A kill switch cleared too early** let an old runner collide with a new campaign. The adapter refused concurrent access and the call cost nothing — but the runner counted it. Both are fixed: a global runner lock, and refusals that never reached the provider are not counted against the budget.
- **We invalidated our own campaign** rather than report a flatline as a finding. Eleven calls. It was the right call.

Reproducing

```
# construct (zero calls, awaits explicit approval)
python new_ablation_campaign.py --fixture medi-ny-parser-rework-v1 `
--arms "vanilla,vanilla-loop,standard-loop" --output <campaign.json>
```

```
# approve an exact call count, then run
python approve_ablation_campaign.py --campaign <campaign.json> --approved-by "<name>"
--exact-calls 21
python run_ablation_campaign.py --campaign <campaign.json> --max-calls 21
```

Evals/local/STOP halts any campaign before its next call. Campaign records, per-run manifests, host check suites, loop reports and grader outputs are preserved under Evals/runs/ and the candidate package's evidence/.